arXiv:2607.27674v1 [cs.SE] 30 Jul 2026

Highlights

Can Large Language Models Resolve Real Java Merge Conflicts? An Evaluation with a Calibrated LLM-as-Judge

Bowen Shen

- We evaluate modern LLMs on real Java merge conflicts from Conflict-Bench using a two-layer harness that strictly separates a no-ground-truth solver from a ground-truth evaluator.
- The developer-match LLM-as-judge is validated against human labels *first* (precision 100%, recall 64.6%, n=292), so every reported rate is a conservative lower bound.
- Under the validated judge, LLMs match the developer resolution on ≈55% of true conflicts and beat the strongest traditional tool by 18–22 points under a coverage-fair convention.
- The LLM's advantage is coverage, not accuracy: structured tools abstain on 20–90% of conflicts; the LLM under forced resolution abstains on none.
- A deterministic structural check catches invalid code the LLM judge waved through (① accepted 4 of 5 structural failures), showing structural correctness must not be judged by an LLM.

Can Large Language Models Resolve Real Java Merge Conflicts? An Evaluation with a Calibrated LLM-as-Judge

Bowen Shen[a]

[a] *Virginia Polytechnic Institute and State University, Blacksburg, VA, 24060, USA*

Abstract

Merge conflicts are a recurring cost of collaborative software development, and the traditional structured and semi-structured merge tools that address them frequently *abstain*: when their heuristics do not apply, they leave the conflict unresolved. Large language models (LLMs) can instead produce a candidate resolution for essentially any conflict, but measuring whether those resolutions are actually *good* at scale is itself hard, because obtaining human desirability judgments for every model output does not scale. This paper studies both problems together on real Java merge conflicts drawn from ConflictBench. We first build an LLM solver as a *generate–validate–retry* agent that uses only inference-time signals (conflict markers, a Java parser, and duplicate-declaration checks) and never sees the developer's answer. We then evaluate its resolutions with a two-metric suite: (1) a *developer-match* LLM-as-judge implemented as a G-Eval metric and, crucially, *calibrated against ConflictBench's human labels before being used*, and (2) a deterministic *structural-validity* check that does not use an LLM. On a meta-evaluation of n=292 human-labeled cases, the judge achieves **100% precision (zero false accepts)** at 64.6% recall, so every acceptance it emits is trustworthy and every downstream rate is a conservative lower bound. Under this validated judge, LLM solvers match the developer's own resolution on **≈55%** of true conflicts (conservative floor), and under a coverage-fair comparison the LLMs (55–59%) beat the strongest traditional tool (AutoMerge, 36.7%) by roughly 18–22 points—an edge that comes almost entirely from *coverage*, not raw accuracy: the tools abstain on 20–90% of conflicts while the LLM under forced resolution abstains on none. Finally, we find the LLM judge accepted 4 of the 5 resolutions that fail the deter-

Email address: bowenshe@vt.edu (Bowen Shen)

ministic structural check, concrete evidence that structural correctness must *not* be delegated to an LLM. We release the harness as an open engineering artifact.

Keywords: software merge, merge conflict resolution, large language models, LLM-as-a-judge, empirical software engineering, evaluation

---

1. Introduction

In collaborative software development, programmers create branches to add features and fix bugs in parallel, and later merge those branches to integrate their edits. When two branches edit the same region of a file in divergent ways, the edits *conflict* and cannot be co-applied automatically. Resolving these conflicts is a well-documented drain on developer time, and a large body of work has built structured and semi-structured merge tools to detect and resolve them automatically [1, 2, 3, 4].

A defining property of these traditional tools is that they *abstain* whenever their heuristics do not apply. A structured merge tool that cannot align two abstract syntax trees, or a line-based tool that sees genuinely overlapping edits, simply leaves the conflict markers in place and reports an unresolved conflict. Abstention is safe, but it means the tool contributes nothing on exactly the cases a developer most wants help with. Our earlier benchmark study, ConflictBench, quantified this: across five state-of-the-art tools, applicability varied widely and no tool resolved more than a fraction of real conflicts [5].

Large language models (LLMs) change the shape of the problem. An LLM prompted with a conflict will almost always *produce* a candidate resolution, whether or not the conflict is "easy." This raises two questions that this paper addresses together:

Q1. Can we trust an automatic judge? Human desirability judgments do not scale to every model output, so we need an automatic evaluator. But an LLM-as-judge is only useful if we know how far to trust it. *How accurately does a calibrated LLM-as-judge reproduce human desirability labels on merge-conflict resolutions?*

Q2. How good are LLM resolutions? *Under that validated judge, how often do modern LLMs match the developer's own resolution on real Java conflicts, and how does that compare to traditional merge tools once we account for abstention?*

We answer these on real Java conflicts from ConflictBench with a harness we call *ConflictAgent*. Our design rests on a strict two-layer separation (Fig. 1). The *solver layer* is an agent that generates a candidate, validates it with inference-time signals only, and retries on failure—it never sees the developer's answer or any judge verdict. The *evaluation layer* runs afterward and is allowed to use ground truth: it compares the candidate against the developer's actual resolution using two orthogonal metrics.

The key methodological move is that we *calibrate the judge before we use it*. We first treat ConflictBench's recorded tool outputs and their human desirability labels as ground truth, ask the judge to grade those same tool outputs, and measure how well it reproduces the human labels. Only after the judge passes this meta-evaluation do we use it to grade LLM resolutions. On n=292 human-labeled cases the judge achieves **100% precision** (zero false accepts) at 64.6% recall. The zero-false-accept property is what makes the rest of the study interpretable: when the judge says a resolution is acceptable, it is trustworthy; the cost is that the judge under-credits acceptable alternatives, so every rate we report is a *conservative lower bound*.

Contributions.. This paper makes the following contributions:

1. A two-layer evaluation harness for LLM merge-conflict resolution that strictly separates a no-ground-truth solver agent from a ground-truth evaluator (Sec. 3).
2. A developer-match LLM-as-judge that is *validated against human labels before use* (precision 100%, recall 64.6%, n=292), turning every downstream number into a defensible lower bound (Sec. 4).
3. An evaluation showing that LLMs match the developer resolution on ≈55% of true conflicts and, under a coverage-fair convention, beat the strongest traditional tool by 18–22 points—an edge driven by coverage rather than accuracy (Sec. 5).
4. Evidence that structural correctness must be judged deterministically: the LLM judge accepted 4 of the 5 resolutions that fail a deterministic parse/structure check (Sec. 5.3).

We frame this work as an *evaluation study* built on an open engineering artifact, not as a new benchmark: the data, developer resolutions, human labels, and five tool baselines all come from ConflictBench [5]; our contribution is the LLM agent and the calibrated evaluation methodology around it.

2. Background and Related Work

2.1. Three-way merge and conflicts

Version-control systems merge branches with a three-way `diff3` algorithm that compares each branch against the common ancestor (*base*) [7]. When both branches change overlapping lines, `diff3` emits conflict markers (`<<<<<<<`, `|||||||`, `=======`, `>>>>>>>`) delimiting the left, base, and right variants. Following ConflictBench, we distinguish *true* conflicts (the two sides make genuinely incompatible changes that require human judgment) from *false* conflicts (textually overlapping but logically compatible edits that a good tool should merge automatically) [5]. A prior characterization study found that developers resolve a large fraction of true textual conflicts by keeping one branch's edits wholesale [6], which is part of why simple side-picking baselines are a meaningful comparison point (Sec. 3.3).

2.2. Automatic merge tools

Line-based tools such as KDiff3 [8] operate purely on text. Semi-structured and structured merge tools—FSTMerge [1], JDime [2], IntelliMerge [3], and AutoMerge [4]—exploit the syntax and, in some cases, the refactoring history of the program to merge more precisely. Their common limitation, quantified in ConflictBench [5], is coverage: when a tool's structural assumptions do not hold it abstains, leaving the conflict unresolved. We reuse exactly these five tools and their recorded outputs as baselines.

2.3. LLMs for code and for merge resolution

LLMs have become strong at code generation and program repair [9, 10], and prior learning-based work has targeted merge conflicts directly; MergeBERT, for example, framed resolution as a classification over a fixed set of edit patterns [11]. Our focus is different: rather than train a model, we evaluate how well modern general-purpose LLMs resolve real conflicts under an honest, calibrated measurement protocol, and we compare them to traditional tools on the same judged scale.

2.4. LLM-as-a-judge

Using an LLM to evaluate open-ended outputs is now common [12], and G-Eval showed that a chain-of-thought prompt with a form-filled, probability-weighted score aligns better with humans than a single binary verdict [13]. The recognized risks are self-preference (a model favoring its own family's outputs) and miscalibration. We address the first by making the judge a *different vendor* from both solvers, and the second by *validating the judge*

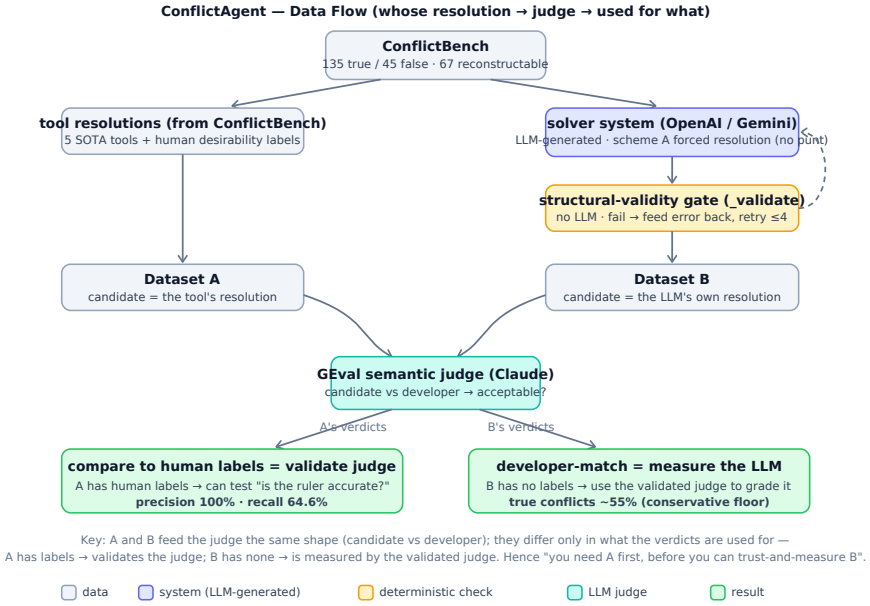


Figure 1: The two-layer data flow. The solver (top) generates candidates without ground truth; the evaluation layer (bottom) uses ground truth. Dataset A grades the *tools'* recorded outputs (which carry human labels) to *validate* the judge; Dataset B then uses the validated judge to *measure* the LLMs. This is why the judge must be calibrated on A before it is trusted on B.

against human labels before using it—the step that distinguishes a trustworthy automatic metric from an unverified one.

3. The ConflictAgent Harness

3.1. Data

We use ConflictBench [5] as our data source. It contains 180 textual merge scenarios sampled from 180 open-source Java projects; each scenario ships the `base`, `left`, `right`, and developer (`child`) versions, the outputs of the five merge tools when available, and human labels including per-tool desirability judgments. ConflictBench labels 136 of the 180 conflicts as true and 44 as false. One scenario (`orientdb@501dac79`) is a line-ending artifact that `git merge` resolves cleanly after newline normalization; we treat it as a false conflict, giving an effective 135/45 split. This reclassification has no effect on any reported number, since the scenario is already excluded upstream as a non-conflict.

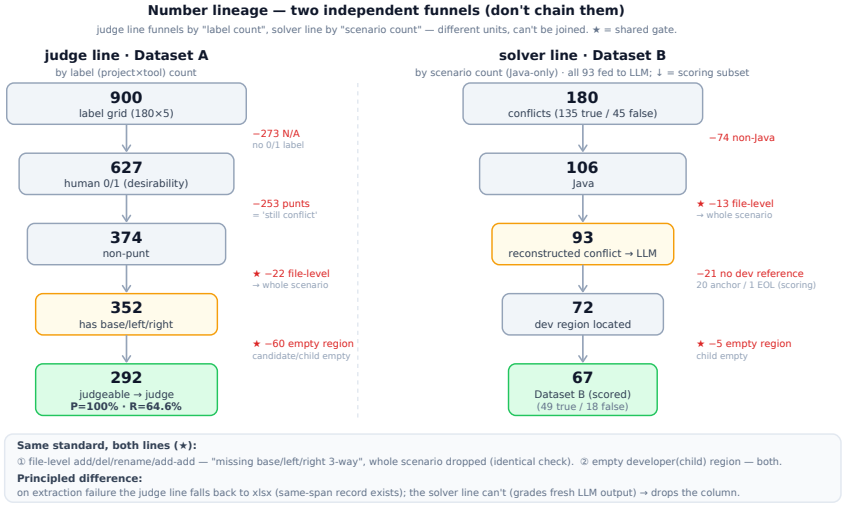


Figure 2: Two independent number funnels that must *not* be chained: the judge line (left) is counted in labeled (scenario × tool) pairs and ends at the n=292 meta-evaluation set; the solver line (right) is counted in Java scenarios and ends at the 93 reconstructable conflicts fed to each LLM. Shared structural gates are marked ⋆.

Because our structural validity check is Java-specific and requires a reconstructable in-file three-way merge, our solver-evaluation scope is a subset of the full benchmark (Fig. 2, right). Of the 180 scenarios, 106 are Java; of those, 93 are reconstructable from complete base/left/right files. All 93 are fed to each LLM; the smaller numbers reported later are *scoring* subsets (cases we can reliably grade), not input samples.

3.2. The solver agent (no ground truth)

The solver layer (Fig. 3) turns one scenario into one candidate resolution through a generate–validate–retry loop:

1. Reconstruct a `diff3` conflict file from base/left/right with `git merge-file --diff3`.
2. Select the target conflict block by matching content from ConflictBench's annotated merged snippet against the reconstructed blocks (files may contain several blocks).
3. Build a *windowed* prompt: the package/import/type skeleton plus the smallest complete brace scope enclosing the target block, with the rest of the file elided. Files below a size threshold are shown whole. Win-

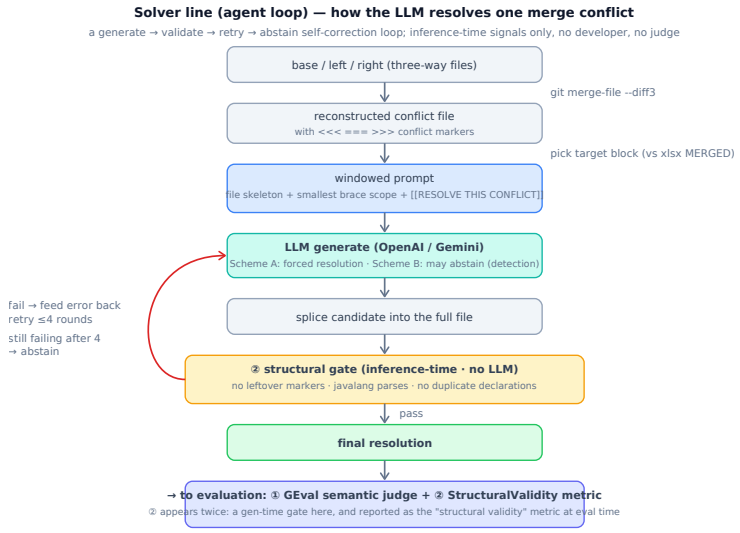


Figure 3: The solver agent loop: reconstruct the diff3 file, window the prompt around the target block, generate a resolution, splice it into the full file, and run the deterministic structural gate (no leftover markers, parses via `javalang`, no duplicate declarations). Validation failures feed the error back for up to four retries; persistent failure abstains. The loop uses inference-time signals only—no developer answer, no judge.

dowing affects only what the model sees; validation and splicing always use the full reconstructed file.

4. Ask the solver to replace *only* the tagged conflict region.
5. Splice the candidate back into the full file.
6. Validate using inference-time signals only: no leftover conflict markers, the spliced file parses as Java via `javalang` [14], and no duplicate declarations (which catch over-scoped output).
7. Retry up to four times when validation fails, feeding the validator error back to the model; if it still fails, abstain.

Two prompt schemes are kept as methodology but never pooled. **Scheme A** (the primary, scored scheme) forces the model to produce a resolution plus a self-reported strategy and confidence. **Scheme B** is a detection/robustness variant that first lets the model declare a `TRUE_CONFLICT` and abstain; it is reported as a separate capability, not folded into the Scheme-A headline. The exact prompts are in the released harness.

We evaluate two solvers of different vendors, `gpt-5.4-2026-03-05` (OpenAI) and `gemini-3.5-flash` (Google), both at temperature 0 for repro-

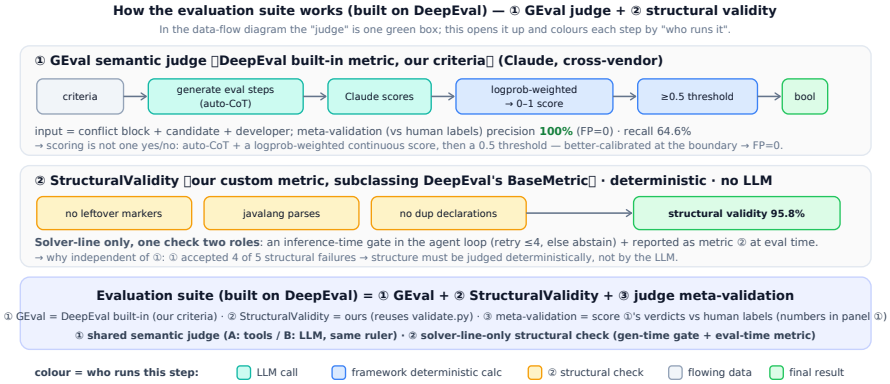


Figure 4: Inside the evaluation suite. Metric ① (developer-match) is a G-Eval LLM judge that generates chain-of-thought evaluation steps and a probability-weighted score before thresholding. Metric ② (structural validity) is a deterministic, non-LLM check. Meta-validation ③ scores ①'s verdicts against human labels. Colors indicate which component runs each step.

ducibility.

3.3. The evaluation suite (ground truth allowed)

Evaluation runs after the agent finishes and is the only place ground truth enters. It is built on DeepEval [15] and comprises two orthogonal metrics (Fig. 4):

① *Developer-match (LLM judge)..* A G-Eval [13] metric (*ResolutionAcceptability*) sees the diff3 conflict block, the candidate resolution, and the developer's actual resolution, and decides whether the candidate is an acceptable *semantic* match (not a verbatim match, which would be far too strict). The judge is `claude-sonnet-4-6`—deliberately a different vendor from both solvers to avoid self-preference—at temperature 0.

② *Structural validity (deterministic, no LLM)..* A candidate is structurally valid if it has no leftover markers, parses as Java, and introduces no duplicate declarations. This reuses the same three checks the agent applies at generation time, but here it is reported as an independent metric. Critically, ② does *not* gate ①: we report them separately precisely so we can ask whether the LLM judge is reliable about structural correctness (Sec. 5.3).

Baselines.. Because a strong first-round solver usually emits syntactically valid output, an early single-shot baseline carried little signal and was dropped. We instead compare against four interpretable trivial baselines—`pick-left`, `pick-right`, `pick-longer`, and `union`—and against the five ConflictBench tools, all scored by the *same* ① judge. A secondary *standalone-valid* judge (is the candidate reasonable from base/left/right alone, without the developer answer?) is retained but is meaningful only for false conflicts, which have an objective mechanical merge; it is deliberately *excluded* from the headline because it has no ground truth to validate against.

4. Calibrating the Judge

Before using the ① judge to grade LLM resolutions, we validate it against ConflictBench's human labels (a *meta-evaluation*). The idea (Fig. 1) is that the tool outputs already carry human desirability labels, so we can ask the judge to grade those same tool outputs and compare its verdicts to the humans'.

Population.. The meta-evaluation funnel (Fig. 2, left) is measured in labeled (scenario $\times$ tool) pairs, not scenarios. It starts from the $180 \times 5 = 900$ label grid and removes, in order: 273 cells with no 0/1 desirability label; 253 tool punts (the tool left the conflict unresolved—a detection event, not a resolution); 22 file-level operations (add/delete/rename, for which there is no reconstructable in-file three-way merge—the same structural gate the solver applies at $106 \rightarrow 93$); and 60 empty regions (candidate or developer empty after extraction). This leaves n=292 **judgeable cases**.

Result.. Against the human labels, at the default acceptance threshold 0.5:

Table 1 reports the headline: **precision 100%** (zero false accepts) at recall 64.6%. The interpretation drives the rest of the paper. A false accept—calling a bad resolution acceptable—is the fatal error for a judge, because it would inflate every downstream rate; the judge makes zero of these on this set. The cost is recall: the judge under-credits some acceptable alternatives (it is stricter than the median human annotator). Consequently every solver or tool rate produced by this judge is a **conservative lower bound**, and an `ACCEPTABLE` verdict can be trusted.

Why high precision is not an artifact of clean inputs.. A residual minority of label pairs record the candidate and developer at inconsistent scopes (e.g., a 2-line developer window versus a 24-line tool block). Such a mismatch can only cause the judge to *reject* a valid resolution—depressing recall—never

Table 1: Judge meta-evaluation against human desirability labels (n=292, judge `claude-sonnet-4-6`, threshold 0.5). The judge has zero false accepts.

Metric	Value
Accuracy	78.1%
Precision	**100.0%**
Recall	64.6%
True positives (TP)	117
False positives (FP)	**0**
True negatives (TN)	111
False negatives (FN)	64

to produce a false accept. Precision therefore stays at 100% regardless, and the conservative-lower-bound conclusion is stable.

Standalone-valid (secondary).. On a separate 40-item human-labeled blind sample, the standalone-valid judge scores accuracy 85%, precision 88.2%, recall 93.8% on the 20 false conflicts (TP15/FP2/TN2/FN1). Only the false-conflict figure is a substantive correctness number, because a true conflict has no context-free correct answer; this metric is retained as supplementary and kept out of the headline.

5. Results

We organize results around Q2. Section 5.1 reports developer-match rates, Section 5.2 compares against traditional tools under a coverage-fair convention, and Section 5.3 reports structural validity and the judge-reliability finding.

Scored population.. Extending the Scheme-A branch of Fig. 2, the 72 comparable scenarios lose 5 with an empty developer region, giving **67 Dataset B scenarios (49 true / 18 false)**. Fed to both providers this yields 132 records (OpenAI 67, Gemini 65—Gemini produced two empty resolutions that cannot be graded), i.e. 96 true and 36 false provider-level records.

5.1. How often do LLMs match the developer?

Table 2 shows the ① developer-match results. On true conflicts—the hard case requiring judgment—the two solvers match the developer's own resolution 55.1% (OpenAI) and 61.7% (Gemini) of the time. We quote the **conservative floor, ≈55%** (the lower provider), rather than the pooled 58.3%,

Table 2: ① Developer-match rate (validated G-Eval judge, threshold 0.5), by conflict type and provider. The headline quotes the conservative floor ≈55% (OpenAI on true conflicts).

Conflict type	OpenAI	Gemini	Pooled
True	27/49 = 55.1%	29/47 = 61.7%	56/96 = 58.3%
False	12/18 = 66.7%	11/18 = 61.1%	23/36 = 63.9%

Table 3: ① LLM vs. five traditional tools on true conflicts (n=49), same judge, coverage-fair. "Among-resolved" scores only cases where the tool produced a resolution; "Overall" counts a punt as a miss (the coverage-fair headline). "Punt" is the count of abstentions.

Resolver	Among-resolved	**Overall**	Punt
LLM Gemini	29/47 = 61.7%	**29/49 = 59.2%**	0
LLM OpenAI	27/49 = 55.1%	**27/49 = 55.1%**	0
AutoMerge	18/38 = 47.4%	18/49 = 36.7%	10
JDime	17/31 = 54.8%	17/49 = 34.7%	16
IntelliMerge	13/27 = 48.1%	13/49 = 26.5%	22
FSTMerge	6/21 = 28.6%	6/49 = 12.2%	18
KDiff3	2/4 = 50.0%	2/49 = 4.1%	45

for two reasons: the providers resolve different subsets so a pooled average is not meaningful, and reporting a floor is consistent with the judge being a strict lower bound. Because the judge counts only developer-*matching* resolutions—not resolutions that are valid but simply different from what the developer chose—this ≈55% is a lower bound in a second sense as well.

5.2. LLMs versus traditional tools: a coverage-fair comparison

The central comparison pits the LLM against the five ConflictBench tools *on the same judged scale*. All six are scored by the same ① judge on the 49 true conflicts of the reconstructable-Java overlap. The fair convention when one side may abstain is to *fix the denominator by the side that always answers* (the LLM under Scheme A) and count the other side's abstention as a miss rather than shrinking the denominator—otherwise a tool's biggest weakness, abstaining, would be hidden.

Table 3 contains the paper's main empirical result. *Among resolved cases*, the LLM and the strongest tool are close (Gemini 61.7% vs. JDime 54.8%): when a structured tool does fire, its accuracy is competitive. The gap opens entirely on **coverage**. The tools abstain heavily—KDiff3 on 45 of 49 conflicts, IntelliMerge on 22, JDime on 16, AutoMerge on 10—while the LLM

Table 4: ② Structural validity (deterministic, no LLM), by conflict type.

Conflict type	Valid	Rate
True	92/96	**95.8%**
False	35/36	97.2%

under Scheme A abstains on none. Under the coverage-fair overall convention the LLMs (55–59%) clear the strongest tool (AutoMerge, 36.7%) by roughly 18–22 points.

The qualitative conclusion is deliberately narrow: the LLM's advantage is *not* "substantially more accurate" but "still produces a gradeable resolution where structured tools give up." That is the applicability claim, and it is the honest reading of Table 3.

5.3. Structural validity, and when not *to trust an LLM judge*

Table 4 shows structural validity is high: 95.8% on true conflicts. The 4.2% that fail are all retries-exhausted cases that never reached a valid resolution within the four-round budget—three `javalang` parse failures and two duplicate-declaration cases.

The important finding is about the relationship between the two metrics. Of the five resolutions that fail the deterministic structural check ②, **four were accepted by the LLM judge** ①: the LLM judge waved through code that does not parse. This is concrete evidence that structural correctness must be evaluated by a deterministic checker, *not* delegated to an LLM, and it is why we keep ② independent of ① rather than letting the judge subsume it. Knowing when *not* to use an LLM is part of using one well.

6. Case Studies

To make the aggregate numbers concrete, we walk through six representative scenarios (drawn from the saved evaluation records) that each illustrate one design decision behind the harness. A caveat on reading them: the per-case verdicts in Table 5 are saved records used to *explain* the evaluation design (why we need trivial baselines, why standalone-valid and developer-match answer different questions, why extraction is guarded); these design points are judge-agnostic. The *reported* rates are the aggregate G-Eval numbers in Section 5; individual per-case entries here are anchors, not the headline figures.

Table 5: Six case studies, each illustrating one evaluation-design point. "T/F" is the ConflictBench human label (true/false conflict). *dev-match* = judged an acceptable match to the developer's resolution; *standalone* = judged reasonable from base/left/right alone; *final-valid* = passed structural validation after splicing. "–" = not applicable (e.g., an excluded scenario or a punt).

Scenario	T/F	Observation (as recorded)	Design point it anchors
`error-prone@6f83c083`	T	Both LLMs match the developer; all four trivial baselines miss (dev-match false).	LLMs beat heuristics on a genuine conflict.
`dubbo@b7b34b6c`	T	Both LLMs match the developer; a side-pick can look plausible in isolation (standalone true) yet miss the developer's intent (dev-match false).	Why we replaced the single-shot baseline with trivial baselines.
`mybatis-3@3502f7ce`	T	Both LLMs miss the developer; the `pick-right` baseline matches it.	Baselines are necessary—LLMs can over-merge.
`RxJava@45c9dc85`	F	Candidates look reasonable standalone but do not match the developer; OpenAI additionally fails structural validation after splice.	Standalone-valid, developer-match, and structural-validity are three different questions.
`proxyee-down@1d9d7f71`	F	Agent produces and validates a resolution, but context anchors are not unique, so the developer region is not extractable.	Guarded extraction: why denominators are smaller than 93 (a data-integrity guard, not a model failure).
`Matisse@93d0051c`	T	Under Scheme B both solvers punt (declare `TRUE_CONFLICT`); the human label agrees.	Detection: a punt is a correct abstention event, not a failed resolution.

The value case (`error-prone`, `dubbo`).. These are true conflicts where every trivial baseline misses the developer's resolution but both LLMs match it. They are the concrete form of the headline: on genuine conflicts an LLM can recover the developer's intent where side-picking and union heuristics cannot.

The anti-overclaim case (`mybatis-3`).. Here the developer simply took one side, and the `pick-right` baseline captures that while both LLMs produce a reasonable-but-different merge. This is exactly why we report trivial baselines alongside the LLM: without them, an LLM that "looks smart" could be losing to a one-line heuristic on the cases that matter.

Three questions, not one (`RxJava`).. Several candidates are reasonable in isolation (*standalone* true) yet are not the developer's answer (*dev-match* false), and OpenAI's candidate additionally fails structural validation after being spliced back (*final-valid* false). This single scenario shows why the suite keeps developer-match, the secondary standalone-valid check, and the

deterministic structural metric as separate axes.

Guarded ground truth (`proxyee-down`).. The agent resolves the conflict, but the evaluation layer refuses to guess the developer's corresponding region when context anchors are not unique, so the scenario is excluded from the developer-match denominator. This is the mechanism behind the conservative $93 \rightarrow 72$ scoring funnel: the guard abstains rather than risk a wrong label.

Abstention as a capability (`Matisse`).. Under the Scheme-B detection variant, both solvers decline to auto-resolve and the human label agrees that the conflict is genuinely true. A punt is scored as a correct detection event, not as a failed resolution—the same abstention that, for the traditional tools, we count as a coverage miss in Table 3.

7. Discussion

Coverage is the story, not accuracy.. The temptation with LLM results is to headline a single accuracy number. Our comparison resists that: among resolved cases the LLM barely edges the best tool. What changes the picture is that structured tools decline to answer most of the time. In practice a developer facing a conflict benefits more from a gradeable proposal on 100% of conflicts than from a marginally better proposal on the 10–80% a tool is willing to touch.

Everything is a lower bound.. Three design choices all push our numbers down rather than up: the judge is strict (100% precision, 64.6% recall), so it under-credits acceptable alternatives; we count only developer-matching resolutions, excluding valid-but-different ones; and we quote the lower-provider floor rather than a pooled average. We prefer to under-claim with a defensible protocol than to over-claim with an unvalidated one.

Calibrate the ruler before you measure with it.. The methodological point generalizes beyond merge conflicts. An LLM-as-judge is a measurement instrument, and an uncalibrated instrument produces uninterpretable numbers. Validating the judge against human labels *first*—and reporting precision, not just accuracy—is what lets us attach a direction (conservative) to every result. We would encourage other LLM-evaluation work to report the judge's own confusion matrix against ground truth before reporting anything the judge measures.

8. Threats to Validity

Construct validity.. The developer-match metric equates "good resolution" with "matches what the developer committed." A different-but-valid resolution is scored as a miss; our numbers are lower bounds accordingly. LLM outputs also lack independent human labels, so residual judge bias cannot be fully excluded—mitigated, but not eliminated, by using a judge from a different vendor than both solvers.

Internal validity.. Developer resolutions are extracted from the `child` file using textual context anchors. When anchors are not unique the scenario is *excluded* rather than guessed; 20 Scheme-A scenarios are excluded this way, splitting into four causes (boundary edges 8, duplicated boilerplate context 6, adjacent-block markers in multi-block files 4, and developer rewrites that erased the anchor 2). The guard excludes rather than guesses, so the denominator is a conservative subset, not a biased one. Separately, `javalang` checks syntax, not full compilation, and for multi-block files still carrying other blocks' markers the structural check degrades to a marker-only check.

External validity.. The study is Java-only and, for the solver evaluation, restricted to the 93 reconstructable scenarios (67 in Dataset B); results may not transfer to other languages or to conflicts outside ConflictBench's sampling. The tool baselines use ConflictBench's recorded outputs rather than locally re-run tools. Finally, LLMs evolve quickly; the specific rates are tied to the two 2026-era models evaluated, though the methodology is model-agnostic.

9. Conclusion

We studied whether modern LLMs can resolve real Java merge conflicts, and—just as importantly—how to measure that credibly. By calibrating an LLM-as-judge against human labels before using it (precision 100%, recall 64.6%, n=292), we turned every downstream rate into a conservative lower bound. Under that judge, LLMs match the developer's own resolution on ≈55% of true conflicts and, under a coverage-fair comparison, beat the strongest traditional merge tool by 18–22 points—an advantage that comes from coverage, since structured tools abstain on much of the workload while the LLM does not. We also showed that structural correctness must stay a deterministic check: the LLM judge accepted four of five resolutions that fail to parse. The harness is released as an open engineering artifact to support replication and extension to other languages and models.

Data and Code Availability

The evaluation harness is available at `https://github.com/UBOWENVT/ConflictAgent`. It builds on ConflictBench [5], which supplies the data, developer resolutions, human labels, and tool baselines.

References

[1] S. Apel, J. Liebig, B. Brandl, C. Lengauer, C. Kästner, Semistructured merge: rethinking merge in revision control systems, in: Proc. 19th ACM SIGSOFT Symp. and 13th European Conf. on Foundations of Software Engineering (ESEC/FSE), 2011, pp. 190–200. doi:10.1145/2025113.2025141.

[2] S. Apel, O. Leßenich, C. Lengauer, Structured merge with auto-tuning: balancing precision and performance, in: Proc. 27th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE), 2012, pp. 120–129. doi:10.1145/2351676.2351694.

[3] B. Shen, W. Zhang, H. Zhao, G. Liang, Z. Jin, Q. Wang, IntelliMerge: a refactoring-aware software merging technique, Proc. ACM Program. Lang. 3 (OOPSLA) (2019) 170:1–170:28. doi:10.1145/3360596.

[4] F. Zhu, F. He, Conflict resolution for structured merge via version space algebra, Proc. ACM Program. Lang. 2 (OOPSLA) (2018) 166:1–166:25. doi:10.1145/3276536.

[5] B. Shen, N. Meng, ConflictBench: a benchmark to evaluate software merge tools, Journal of Systems and Software 214 (2024) 112084. doi:10.1016/j.jss.2024.112084.

[6] B. Shen, M. A. Gulzar, F. He, N. Meng, A characterization study of merge conflicts in Java projects, ACM Trans. Softw. Eng. Methodol. 32 (2) (2023) 40:1–40:28. doi:10.1145/3546944.

[7] S. Khanna, K. Kunal, B. C. Pierce, A formal investigation of Diff3, in: FSTTCS 2007: Foundations of Software Technology and Theoretical Computer Science, Vol. 4855 of LNCS, Springer, 2007, pp. 485–496. doi:10.1007/978-3-540-77050-3_40.

[8] J. Eibl, KDiff3 — file and directory comparison and merge tool, `https://invent.kde.org/sdk/kdiff3`, accessed 2026.

[9] M. Chen, J. Tworek, H. Jun, Q. Yuan, et al., Evaluating large language models trained on code, arXiv:2107.03374 (2021).

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, SWE-bench: can language models resolve real-world GitHub issues?, in: Int. Conf. on Learning Representations (ICLR), 2024.

[11] A. Svyatkovskiy, et al., Program merge conflict resolution via neural transformers, in: Proc. 30th ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE), 2022, pp. 822–833. doi:10.1145/3540250.3549163.

[12] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, et al., Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[13] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, C. Zhu, G-Eval: NLG evaluation using GPT-4 with better human alignment, in: Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 2511–2522.

[14] C. Thunes, javalang: pure Python Java source code parser, `https://github.com/c2nes/javalang`, accessed 2026.

[15] Confident AI, DeepEval: the open-source LLM evaluation framework, `https://github.com/confident-ai/deepeval`, accessed 2026.